Asynchronous JavaScript: Callbacks & XMLHttpRequest

Understanding Asynchronous Operations

JavaScript is single-threaded, meaning it executes code line by line in a sequential manner. However, certain operations like network requests, file reading, or timers take time to complete. If JavaScript waited for these operations to finish before moving to the next line, the entire application would freeze, creating a poor user experience.

Asynchronous programming allows JavaScript to initiate an operation and continue executing other code while waiting for that operation to complete. Once the operation finishes, JavaScript is notified and can handle the result.

Callbacks: The Foundation of Async JavaScript

A callback is a function passed as an argument to another function, which is then invoked inside the outer function to complete an action. Think of it like giving someone your phone number and asking them to call you back when they're done with a task.

Basic Callback Example

Let's start with a simple example that demonstrates the concept:

```
function doSomeTask(till, callback) {
    setTimeout(() => {
        console.log(till);
        callback();
    }, 5000);
}

function showAlert() {
    alert("task completed");
```


Powered by SPRK Technologies

```
}

doSomeTask(10, showAlert);
```

Output:

After 5 seconds:

Console: 10

Alert: "task completed"

In this example, `doSomeTask` accepts two parameters: a value (`till`) and a callback function. After 5 seconds (5000 milliseconds), it logs the value and then executes the callback function. This demonstrates how callbacks allow us to control the flow of asynchronous operations.

Callbacks with Parameters

Callbacks can also receive data from the function that invokes them. This is particularly useful when we need to process the result of an asynchronous operation:

```
function doubleThis(till, callback) {
    setTimeout(() => {
        till = till * 2;
        callback(till);
    }, 5000);
}

function printAns(ans) {
    console.log("Doubling we get:", ans);
}

doubleThis(10, printAns);
```

Output:

After 5 seconds:

Powered by SPRK Technologies

Console: Doubling we get: 20

Here, the callback receives the processed value (20) as a parameter. This pattern is fundamental to handling asynchronous data.

Anonymous Callbacks

Instead of defining a separate function, we can pass an anonymous arrow function directly as a callback. This is common in modern JavaScript:

```
doubleThis(10, (ans) => {
    console.log("Doubling, we get:", ans);
});
```

Output:

After 5 seconds:

Console: Doubling, we get: 20

XMLHttpRequest: The Traditional Approach to HTTP Requests

Before modern APIs like `fetch`, JavaScript used the `XMLHttpRequest` object to make HTTP requests. While it's largely been superseded by newer technologies, understanding XMLHttpRequest provides valuable insight into how asynchronous HTTP communication works and helps appreciate the improvements in modern approaches.

Basic XMLHttpRequest Structure

Here's a reusable function that wraps XMLHttpRequest with callback-based error handling:

```
function get(url, callback) {
    const xhr = new XMLHttpRequest();
```


Powered by SPRK Technologies

```
    xhr.open("GET", url);

    xhr.onload = function() {
        if (xhr.status >= 200 && xhr.status < 300) {
            callback(null, JSON.parse(xhr.responseText));
        } else {
            callback(new Error("Request Failed"), null);
        }
    }

    xhr.onerror = function() {
        callback(new Error("Network error"), null);
    }

    xhr.send();
}
```

Let's break down the key components:

- `new XMLHttpRequest()` creates a new request object
- `xhr.open("GET", url)` initializes the request with the HTTP method and URL
- `xhr.onload` is triggered when the request completes successfully
- `xhr.onerror` handles network-level errors
- `xhr.send()` actually sends the request

The callback follows the error-first callback pattern: the first parameter is an error object (or `null` if successful), and the second is the data.

JSONPlaceholder: A Testing API

For learning and testing purposes, we'll use JSONPlaceholder, a free fake REST API. It provides endpoints for common resources like users, posts, comments, and more, making it perfect for practicing HTTP requests without needing to set up a backend.

Common endpoints include:

- `/users` - Fake user data
- `/posts` - Blog post data
- `/comments` - Comment data

The Problem: Callback Hell

When dealing with multiple asynchronous operations that depend on each other, callbacks can become deeply nested, creating what's known as "callback hell" or the "pyramid of doom." This makes code difficult to read, maintain, and debug.

Let's add a delay function to simulate real-world latency:

```
function delayedGet(url, delay, callback) {
    setTimeout(() => {
        get(url, callback);
    }, delay);
}
```

Now, let's fetch a user, then their posts, then comments on the first post. Each request depends on data from the previous one:

```
delayedGet("https://jsonplaceholder.typicode.com/users/1", 2000,
    (err, data) => {
        if (err) {
            console.error(err);
            return;
        }
        console.log("User: ", data);

        delayedGet("https://jsonplaceholder.typicode.com/posts?userId=1", 2000,
            (err, posts) => {
                if (err) {
                    console.error(err);
                    return;
                }
                console.log("Post:", posts[0]);

                delayedGet("https://jsonplaceholder.typicode.com/comments?postId=1", 2000,
                    (err, comments) => {
                        if (err) {
                            console.error(err);
                            return;
                        }
                        console.log("Comment:", comments[0]);
```


Powered by SPRK Technologies

```
                    });
            });
    });
```

Output:

After 2 seconds:

User: {id: 1, name: "Leanne Graham", username: "Bret", ...}

After 4 seconds total:

Post: {userId: 1, id: 1, title: "sunt aut facere...", ...}

After 6 seconds total:

Comment: {postId: 1, id: 1, name: "id labore ex...", ...}

Issues with Callback Hell

This nested structure presents several problems:

- **Readability:** The code becomes increasingly difficult to follow as nesting increases
- **Maintainability:** Adding or modifying steps requires careful navigation through nested callbacks
- **Error Handling:** Each level needs its own error handling, leading to repetitive code
- **Debugging:** Stack traces become complex and harder to interpret

The Path Forward

The callback hell problem led to the development of better asynchronous patterns in JavaScript. In the next section, we'll explore Promises, which provide a cleaner way to handle asynchronous operations, followed by async/await syntax that makes asynchronous code look and behave more like synchronous code.

These modern approaches maintain the power of asynchronous programming while dramatically improving code readability and maintainability.

Powered by SPRK Technologies

Continue to Part 2: Promises, Fetch API & Async/Await